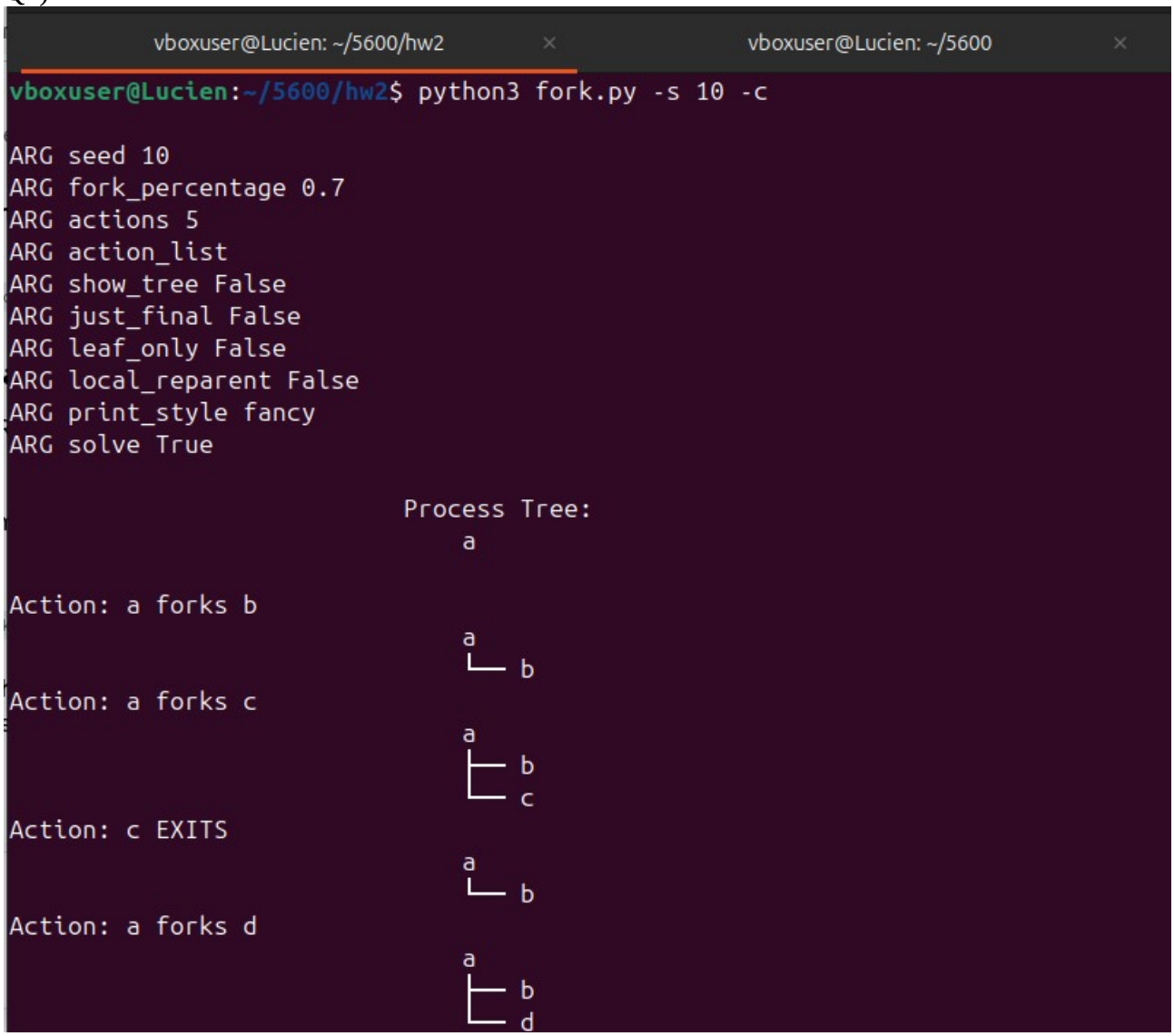
Lucien Roberson
HW2
Chapters 5,6,7

https://github.com/Revo-Lucien/cs5600

Chapter 5:

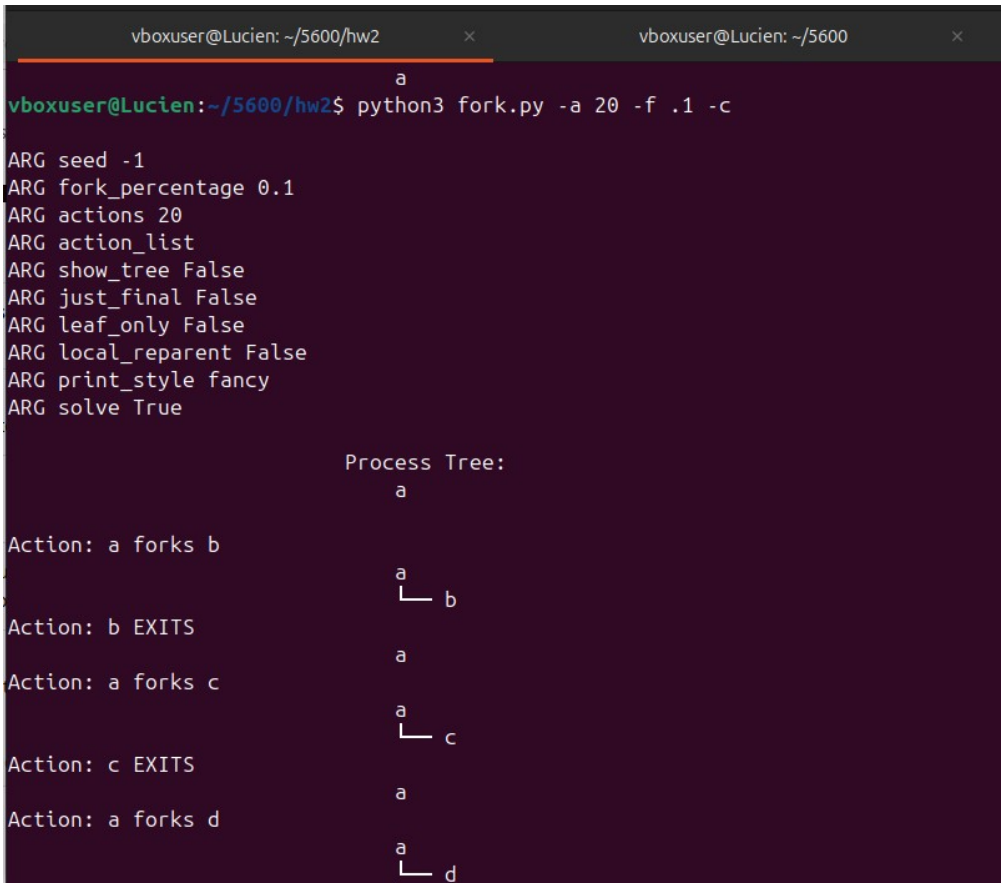
Simulation

Q1)

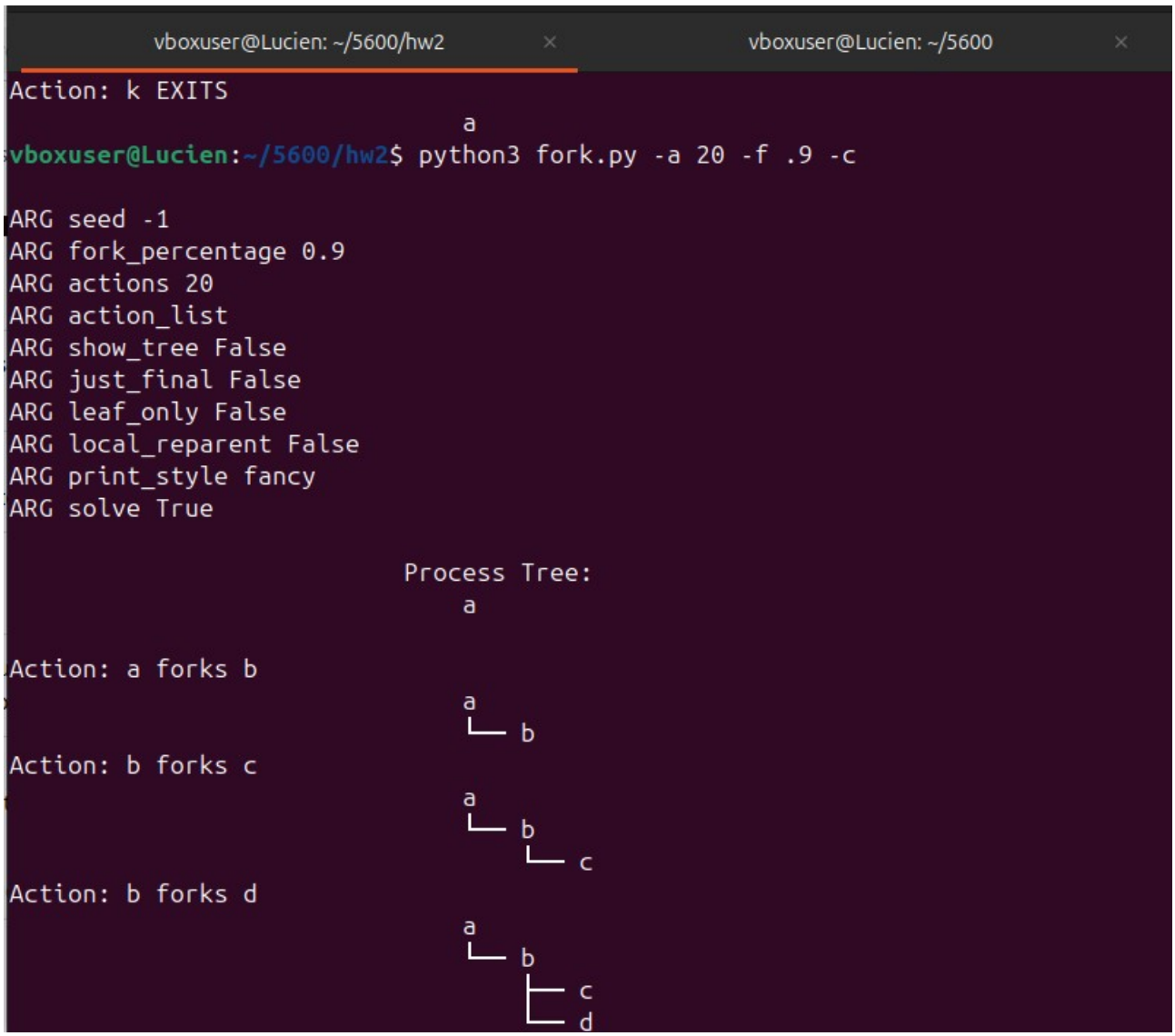


This is a random call that shows a forking several children at random.
I can predict what the tree will look like at each step.

Q2)

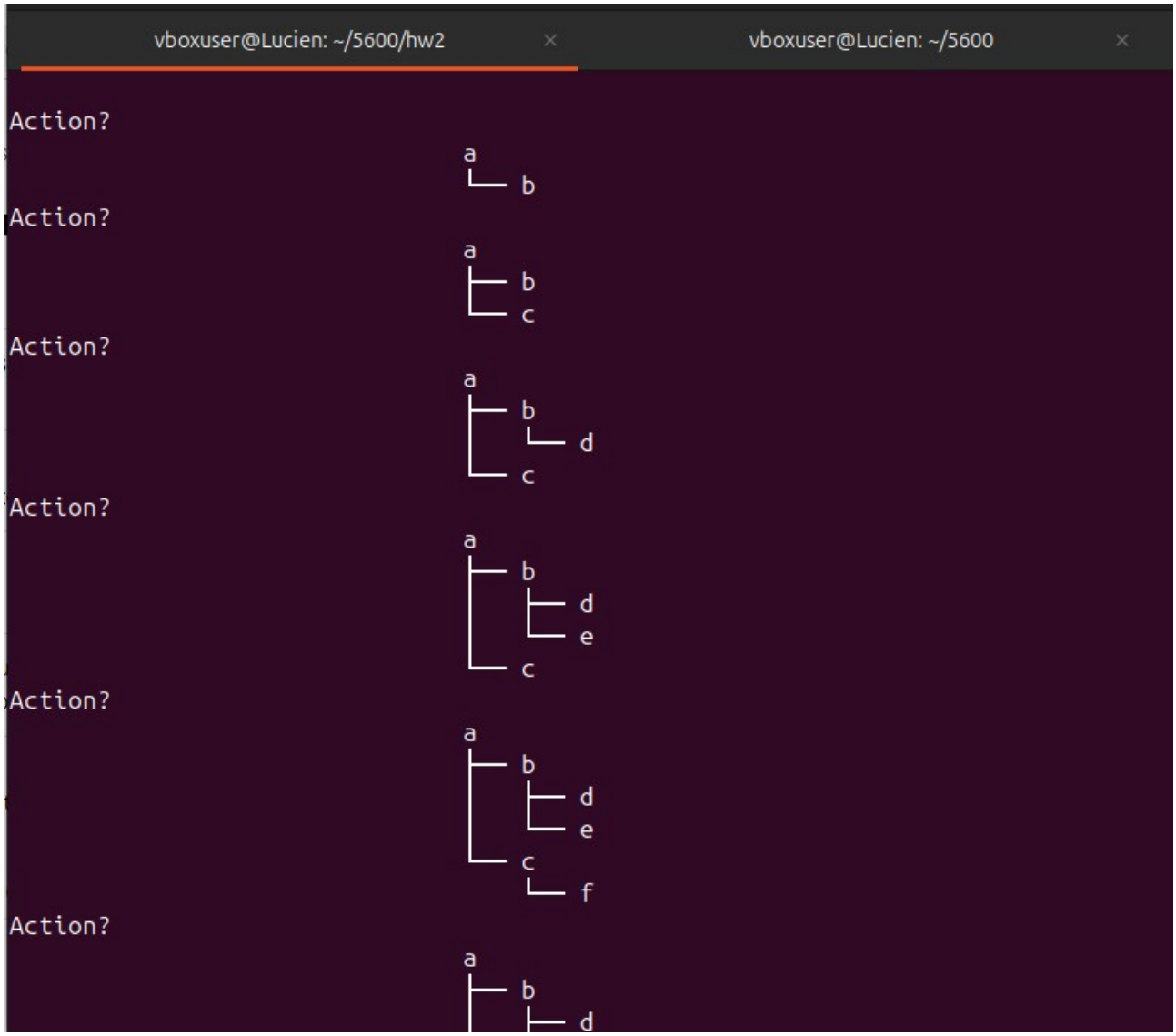


At a low fork-percentage most of the children are terminated before they call further processes, thus a shallow tree.



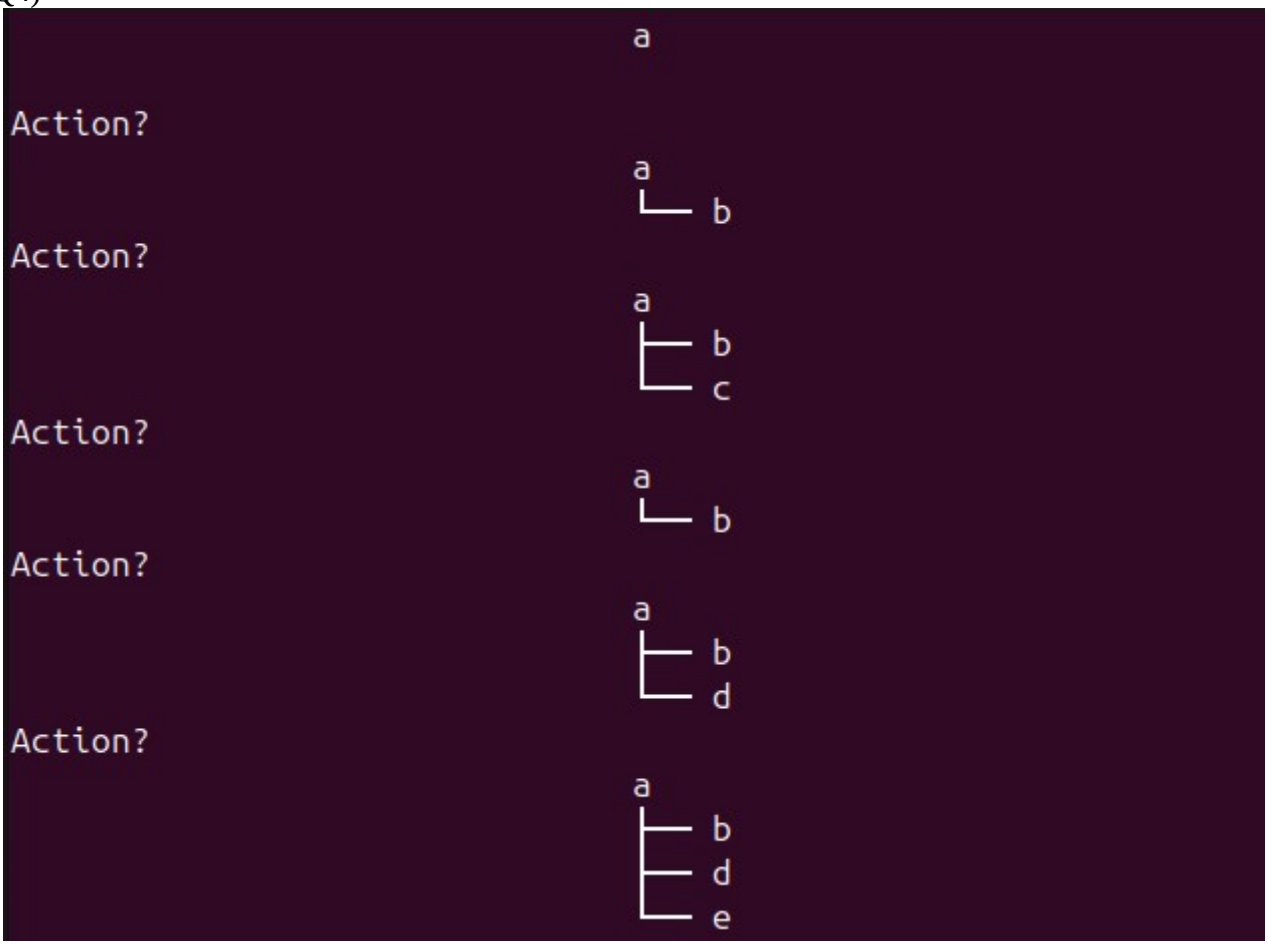
The tree gets a lot deeper as more processes are likely to call their own child before exiting.

Q3)

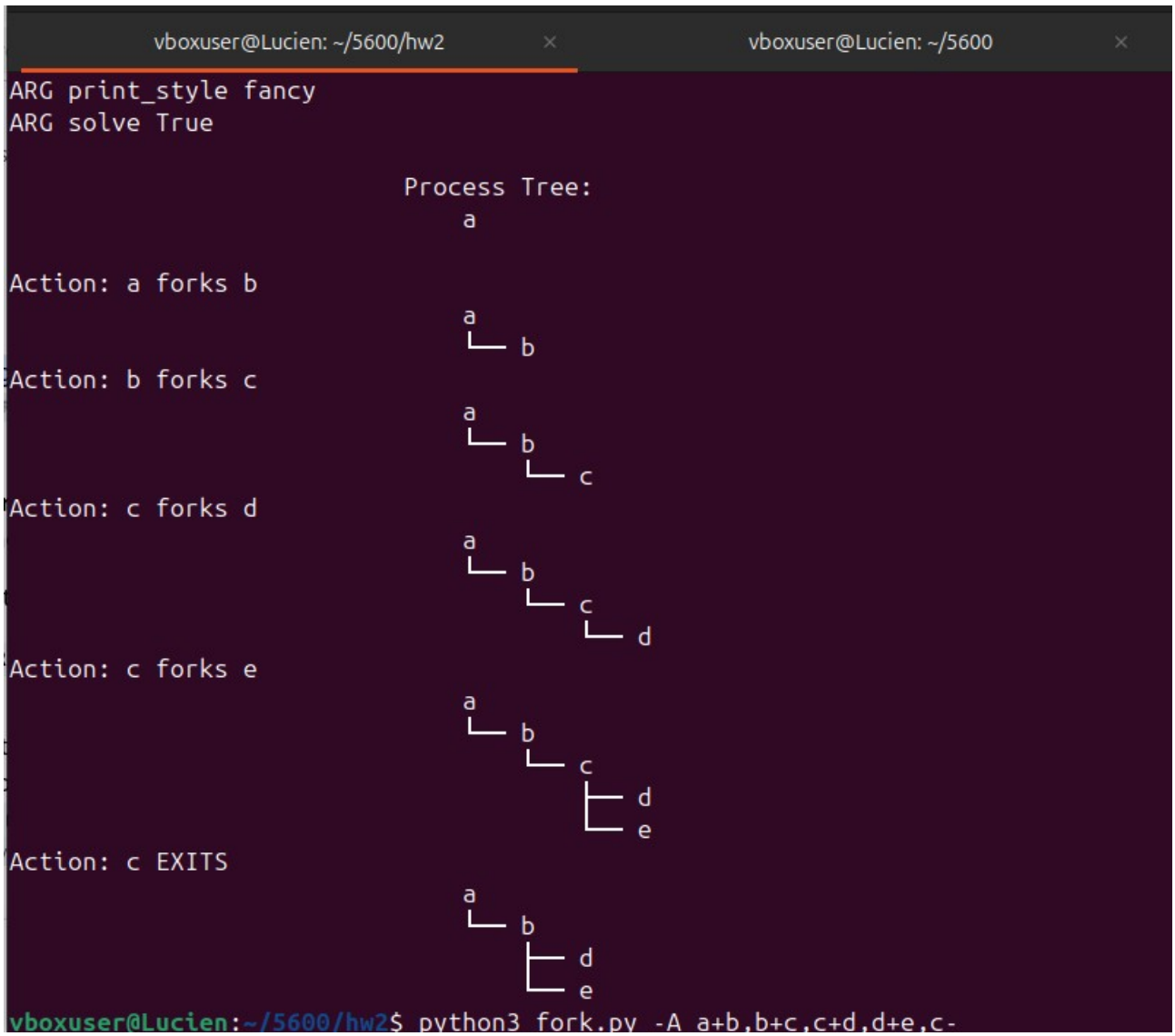


You can tell the actions taken when a child is forked underneath the parent or if a process exits.

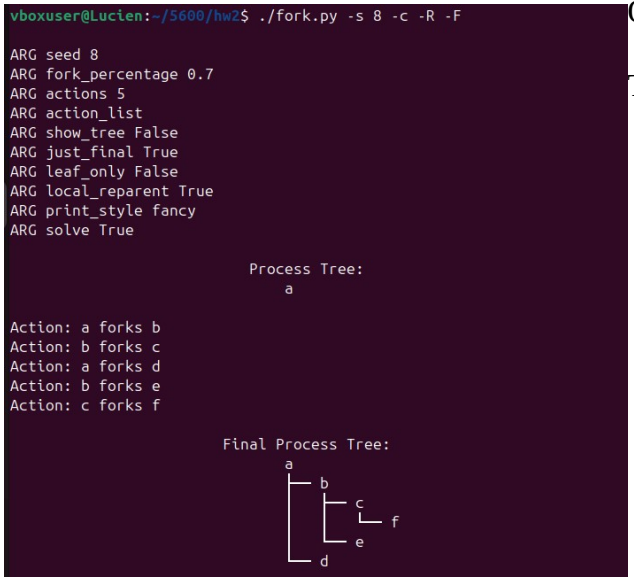
Q4)



This is the output from ./fork.py -A a+b,b+c,c+d,c+e,c- . The orphaned children



When the -R flag is included it acts more like a linked list where the parent node just gets attached immediately to the next node in the list.
After looking up how an OS manages re-parenting I found that it can choose between local and global reparenting with the -R flag.



Q5)

This output matches what it said it would do.

Q6)
I ran the same command as in Q5 with the -t flag.
On the second part of the question, most of the time you wouldn't be able to tell the actions taken unless if none of the children had returned or if there were a very limited amount of calls.

Code

Q1)

vboxuser@Lucien: ~/5600/hw2 × vboxuser@Lucien: ~/5600 ×

GNU nano 7.2 ch5q1.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
        int x= 0;
        int rc = fork();
        if (rc == 0) {
                x = 1;
                printf("Child is running. x is equal to %d \n", x);
        } else {
                x = 2;
                printf("Parent is running. x is equal to %d \n", x);
        }
        return 0;
}
```

[Read 16 lines]

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line

```
vboxuser@Lucien:~/5600/hw2$ gcc ch5q1.c -o ch5q1
vboxuser@Lucien:~/5600/hw2$ ./ch5q1
Parent: x starts as 100
Child: x starts as 100
Child: x is now 1
Parent: x is now 2
Parent: x is still 2 after child exits
```

The output shows that both the child and parent have their own version of the var x and that they can change without interfering with the other's address space.

Q2)

```c
#include <unistd.h>
#include <fcntl.h>

int main() {
        int fd = open("ch5q2.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
        int rc = fork();
        if (rc == 0) {
                write(fd, "child\n", 6);
        } else {
                write(fd, "parent\n", 7);
        }
        return 0;
}
```

Here is the code

```
parent
child
```

Here is the output file that it writes to. It shows that both processes were able to access and write to the file. The kernel has something called the offset that allows processes to write to a file contiguously instead of interweaving.

Q3)

```c
#include <stdio.h>
#include <unistd.h>

int main() {
        int fds[2];
        pipe(fds);
        if (fork() == 0) {
                printf("hello\n");
                fflush(stdout);
                write(fds[1], "x", 1);
        } else {
                char c;
                read(fds[0], &c, 1);
                printf("goodbye\n");
        }
        return 0;
}
```

This is the code followed by the output. The parent can just write to a buffer that will wait until the child writes to it.

```
vboxuser@Lucien:~/5600/hw2$ gcc ch5q3.c -o ch5q3
vboxuser@Lucien:~/5600/hw2$ ./ch5q3
hello
goodbye
```

Q4)

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
        char *args[] = {"ls", "-l", NULL};
        char *envp[] = {NULL};

        for (int i = 0; i < 6; i++) {
                if (fork() == 0) {
                        printf("--- variant %d ---\n", i + 1);
                        fflush(stdout);
                        switch (i) {
                        case 0: execl("/bin/ls", "ls", "-l", (char *)NULL); break;
                        case 1: execle("/bin/ls", "ls", "-l", (char *)NULL, envp); break;
                        case 2: execlp("ls", "ls", "-l", (char *)NULL); break;
                        case 3: execv("/bin/ls", args); break;
                        case 4: execvp("ls", args); break;
                        case 5: execvpe("ls", args, envp); break;
                        }
                        perror("exec failed");   // only reached if exec fails
                        return 1;
                }
                wait(NULL);
        }
        return 0;
```

```
vboxuser@Lucien:~/5600/hw2$ gcc ch5q4.c -o ch5q4
vboxuser@Lucien:~/5600/hw2$ ./ch5q4
--- variant 1 ---
total 584
-rwxrwxr-x 1 vboxuser vboxuser  16048 Oct 18 14:38 ch5q1
-rw-rw-r-- 1 vboxuser vboxuser    529 Oct 18 14:38 ch5q1.c
-rwxrwxr-x 1 vboxuser vboxuser  16040 Oct 18 14:43 ch5q2
-rw-rw-r-- 1 vboxuser vboxuser    252 Oct 18 14:42 ch5q2.c
-rw-r--r-- 1 vboxuser vboxuser     13 Oct 18 14:43 ch5q2.txt
-rwxrwxr-x 1 vboxuser vboxuser  16264 Oct 18 14:50 ch5q3
-rw-rw-r-- 1 vboxuser vboxuser    244 Oct 18 14:49 ch5q3.c
-rwxrwxr-x 1 vboxuser vboxuser  16480 Oct 18 14:57 ch5q4
-rw-rw-r-- 1 vboxuser vboxuser    709 Oct 18 14:56 ch5q4.c
-rwxrwxr-x 1 vboxuser vboxuser  12111 Sep 27 22:12 fork.py
-rw-rw-r-- 1 vboxuser vboxuser 495046 Oct 18 14:48 Lucien_HW_2.odt
--- variant 2 ---
total 584
-rw-rw-r-- 1 vboxuser vboxuser 495046 Oct 18 14:48 Lucien_HW_2.odt
-rwxrwxr-x 1 vboxuser vboxuser  16048 Oct 18 14:38 ch5q1
-rw-rw-r-- 1 vboxuser vboxuser    529 Oct 18 14:38 ch5q1.c
-rwxrwxr-x 1 vboxuser vboxuser  16040 Oct 18 14:43 ch5q2
-rw-rw-r-- 1 vboxuser vboxuser    252 Oct 18 14:42 ch5q2.c
-rw-r--r-- 1 vboxuser vboxuser     13 Oct 18 14:43 ch5q2.txt
-rwxrwxr-x 1 vboxuser vboxuser  16264 Oct 18 14:50 ch5q3
```

All of the variations outputted the same. There are so many variants just because the inputs are a little different merely at the sake of convenience.

Q5)

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
        int rc = fork();
        if (rc == 0) {
                int w = wait(NULL);
                printf("child (pid %d): wait() returned %d\n", getpid(), w);
        } else {
                int w = wait(NULL);
                printf("parent: wait() returned %d (child pid was %d)\n", w, rc);
        }
        return 0;
}
```

Here is my code and output.

The parent returned the PID of its child once the child had exited. Meanwhile the child returned -1 immediately because it wasn't waiting on anything.

```
vboxuser@Lucien:~/5600/hw2$ ./ch5q5
child (pid 14036): wait() returned -1
parent: wait() returned 14036 (child pid was 14036)
```

Q6)

ch5q6.c × | ch5q7.c | ch5q8.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
        int rc = fork();
        if (rc == 0) {
                printf("child (pid %d) running\n", getpid());
        } else {
                int w = waitpid(rc, NULL, 0);
                printf("parent: waitpid() returned %d (child pid was %d)\n", w,
rc);
        }
        return 0;
}
```

```
vboxuser@Lucien:~/5600/hw2$ gcc ch5q6.c -o ch5q6
vboxuser@Lucien:~/5600/hw2$ ./ch5q6
child (pid 14403) running
parent: waitpid() returned 14403 (child pid was 14403)
```

Here is the code and output for Q6. Waitpid lets the parent wait for a specific child giving the programmer more control.

Q7)

~/5600/hw2

ch5q7.c × ch5q8.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
        int rc = fork();
        if (rc == 0) {
                close(STDOUT_FILENO);
                printf("hello from the child\n");
        } else {
                wait(NULL);
                printf("parent: child is done\n");
        }
        return 0;
}
```

```
vboxuser@Lucien:~/5600/hw2$ gcc ch5q7.c -o ch5q7
vboxuser@Lucien:~/5600/hw2$ ./ch5q7
parent: child is done
```

Here is the output for Q7. The child printf doesn’t output anything.

Q8)

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
        int fds[2];
        pipe(fds);                          // fds[0] = read end, fds[1] = write end

        if (fork() == 0) {
                // child 1: writer
                dup2(fds[1], STDOUT_FILENO);   // stdout now goes into the pipe
                close(fds[0]);
                close(fds[1]);
                execlp("echo", "echo", "hello through the pipe", (char *)NULL);
        }

        if (fork() == 0) {
                // child 2: reader
                dup2(fds[0], STDIN_FILENO);    // stdin now comes from the pipe
                close(fds[0]);
                close(fds[1]);
                execlp("cat", "cat", (char *)NULL);
        }

        // parent: close both ends so the reader sees end-of-file
        close(fds[0]);
        close(fds[1]);
        wait(NULL);
        wait(NULL);
        return 0;
}
```

```
vboxuser@Lucien:~/5600/hw2$ gcc ch5q8.c -o ch5q8
vboxuser@Lucien:~/5600/hw2$ ./ch5q8
hello through the pipe
```

The first child piped to the pipe through stdout, the second child piped from the pipe through stdin!

Chapter 6:

There are two c programs too big to paste in this sheet, I have put them in the github. I can paste the output though:

```
vboxuser@Lucien:~/5600/hw2$ gcc syscall.c -o syscall
./syscall
back-to-back gettimeofday: 0 us apart
total: 592189 us, per syscall: 0.5922 us (592.2 ns)
```

```
vboxuser@Lucien:~/5600/hw2$ gcc syscall.c -o syscall
./syscall
back-to-back gettimeofday: 0 us apart
total: 576153 us, per syscall: 0.5762 us (576.2 ns)
```

I instead called a 0-byte read() one million times and divided the total time by the iteration count. Two runs gave about 592 ns and 576 ns per system call, so I estimate the cost of a system call at roughly 580 ns. This is somewhat higher than typical bare-metal numbers, likely because I ran it in a virtual machine.

Chapter 7:

Q1:

```
vboxuser@Lucien:~/5600/hw2$ ./scheduler.py -p FIFO -l 200,200,200 -c
ARG policy FIFO
ARG jlist 200,200,200

Here is the job list, with the run time of each job:
  Job 0 ( length = 200.0 )
  Job 1 ( length = 200.0 )
  Job 2 ( length = 200.0 )


** Solutions **

Execution trace:
  [ time   0 ] Run job 0 for 200.00 secs ( DONE at 200.00 )
  [ time 200 ] Run job 1 for 200.00 secs ( DONE at 400.00 )
  [ time 400 ] Run job 2 for 200.00 secs ( DONE at 600.00 )

Final statistics:
  Job   0 -- Response: 0.00  Turnaround 200.00  Wait 0.00
  Job   1 -- Response: 200.00  Turnaround 400.00  Wait 200.00
  Job   2 -- Response: 400.00  Turnaround 600.00  Wait 400.00

  Average -- Response: 200.00  Turnaround 400.00  Wait 200.00
```

```
vboxuser@Lucien:~/5600/hw2$ ./scheduler.py -p SJF -l 200,200,200 -c
ARG policy SJF
ARG jlist 200,200,200

Here is the job list, with the run time of each job:
  Job 0 ( length = 200.0 )
  Job 1 ( length = 200.0 )
  Job 2 ( length = 200.0 )


** Solutions **

Execution trace:
  [ time   0 ] Run job 0 for 200.00 secs ( DONE at 200.00 )
  [ time 200 ] Run job 1 for 200.00 secs ( DONE at 400.00 )
  [ time 400 ] Run job 2 for 200.00 secs ( DONE at 600.00 )

Final statistics:
  Job   0 -- Response: 0.00  Turnaround 200.00  Wait 0.00
  Job   1 -- Response: 200.00  Turnaround 400.00  Wait 200.00
  Job   2 -- Response: 400.00  Turnaround 600.00  Wait 400.00

  Average -- Response: 200.00  Turnaround 400.00  Wait 200.00
```

They both give the same result

Q2:

```
vboxuser@Lucien:~/5600/hw2$ ./scheduler.py -p SJF -l 100,200,300 -c
ARG policy SJF
ARG jlist 100,200,300

Here is the job list, with the run time of each job:
  Job 0 ( length = 100.0 )
  Job 1 ( length = 200.0 )
  Job 2 ( length = 300.0 )


** Solutions **

Execution trace:
  [ time   0 ] Run job 0 for 100.00 secs ( DONE at 100.00 )
  [ time 100 ] Run job 1 for 200.00 secs ( DONE at 300.00 )
  [ time 300 ] Run job 2 for 300.00 secs ( DONE at 600.00 )

Final statistics:
  Job   0 -- Response: 0.00  Turnaround 100.00  Wait 0.00
  Job   1 -- Response: 100.00  Turnaround 300.00  Wait 100.00
  Job   2 -- Response: 300.00  Turnaround 600.00  Wait 300.00

  Average -- Response: 133.33  Turnaround 333.33  Wait 133.33
```

```
vboxuser@Lucien:~/5600/hw2$ ./scheduler.py -p FIFO -l 100,200,300 -c
ARG policy FIFO
ARG jlist 100,200,300

Here is the job list, with the run time of each job:
  Job 0 ( length = 100.0 )
  Job 1 ( length = 200.0 )
  Job 2 ( length = 300.0 )


** Solutions **

Execution trace:
  [ time   0 ] Run job 0 for 100.00 secs ( DONE at 100.00 )
  [ time 100 ] Run job 1 for 200.00 secs ( DONE at 300.00 )
  [ time 300 ] Run job 2 for 300.00 secs ( DONE at 600.00 )

Final statistics:
  Job   0 -- Response: 0.00  Turnaround 100.00  Wait 0.00
  Job   1 -- Response: 100.00  Turnaround 300.00  Wait 100.00
  Job   2 -- Response: 300.00  Turnaround 600.00  Wait 300.00

  Average -- Response: 133.33  Turnaround 333.33  Wait 133.33
```

They both do the same thing.

Q3:

```
  [ time 584 ] Run job   2 for 1.00 secs
  [ time 585 ] Run job   2 for 1.00 secs
  [ time 586 ] Run job   2 for 1.00 secs
  [ time 587 ] Run job   2 for 1.00 secs
  [ time 588 ] Run job   2 for 1.00 secs
  [ time 589 ] Run job   2 for 1.00 secs
  [ time 590 ] Run job   2 for 1.00 secs
  [ time 591 ] Run job   2 for 1.00 secs
  [ time 592 ] Run job   2 for 1.00 secs
  [ time 593 ] Run job   2 for 1.00 secs
  [ time 594 ] Run job   2 for 1.00 secs
  [ time 595 ] Run job   2 for 1.00 secs
  [ time 596 ] Run job   2 for 1.00 secs
  [ time 597 ] Run job   2 for 1.00 secs
  [ time 598 ] Run job   2 for 1.00 secs
  [ time 599 ] Run job   2 for 1.00 secs ( DONE at 600.00 )

Final statistics:
  Job   0 -- Response: 0.00  Turnaround 298.00  Wait 198.00
  Job   1 -- Response: 1.00  Turnaround 499.00  Wait 299.00
  Job   2 -- Response: 2.00  Turnaround 600.00  Wait 300.00

  Average -- Response: 1.00  Turnaround 465.67  Wait 265.67
```

RR gives a better response time but a worse turn around time.

Q4:
When the jobs arrive in order of increasing length (shortest first), or when all jobs are the same length. In both cases FIFO's order is already what SJF would pick.

Q5:
When the time slice is at least as long as every job and the jobs are listed shortest first. Then RR never preempts a job before it finishes, so it behaves like FIFO, which matches SJF in that order. Equal-length jobs with a quantum at least that long also work.

Q6:

```
vboxuser@Lucien:~/5600/hw2$ ./scheduler.py -p SJF -l 100,100,100 -c
ARG policy SJF
ARG jlist 100,100,100

Here is the job list, with the run time of each job:
  Job 0 ( length = 100.0 )
  Job 1 ( length = 100.0 )
  Job 2 ( length = 100.0 )


** Solutions **

Execution trace:
  [ time   0 ] Run job 0 for 100.00 secs ( DONE at 100.00 )
  [ time 100 ] Run job 1 for 100.00 secs ( DONE at 200.00 )
  [ time 200 ] Run job 2 for 100.00 secs ( DONE at 300.00 )

Final statistics:
  Job   0 -- Response: 0.00  Turnaround 100.00  Wait 0.00
  Job   1 -- Response: 100.00  Turnaround 200.00  Wait 100.00
  Job   2 -- Response: 200.00  Turnaround 300.00  Wait 200.00

  Average -- Response: 100.00  Turnaround 200.00  Wait 100.00
```

```
vboxuser@Lucien:~/5600/hw2$ ./scheduler.py -p SJF -l 200,200,200 -c
ARG policy SJF
ARG jlist 200,200,200

Here is the job list, with the run time of each job:
  Job 0 ( length = 200.0 )
  Job 1 ( length = 200.0 )
  Job 2 ( length = 200.0 )


** Solutions **

Execution trace:
  [ time   0 ] Run job 0 for 200.00 secs ( DONE at 200.00 )
  [ time 200 ] Run job 1 for 200.00 secs ( DONE at 400.00 )
  [ time 400 ] Run job 2 for 200.00 secs ( DONE at 600.00 )

Final statistics:
  Job   0 -- Response: 0.00  Turnaround 200.00  Wait 0.00
  Job   1 -- Response: 200.00  Turnaround 400.00  Wait 200.00
  Job   2 -- Response: 400.00  Turnaround 600.00  Wait 400.00

  Average -- Response: 200.00  Turnaround 400.00  Wait 200.00
```

```
vboxuser@Lucien:~/5600/hw2$ ./scheduler.py -p SJF -l 300,300,300 -c
ARG policy SJF
ARG jlist 300,300,300

Here is the job list, with the run time of each job:
  Job 0 ( length = 300.0 )
  Job 1 ( length = 300.0 )
  Job 2 ( length = 300.0 )


** Solutions **

Execution trace:
  [ time   0 ] Run job 0 for 300.00 secs ( DONE at 300.00 )
  [ time 300 ] Run job 1 for 300.00 secs ( DONE at 600.00 )
  [ time 600 ] Run job 2 for 300.00 secs ( DONE at 900.00 )

Final statistics:
  Job   0 -- Response: 0.00  Turnaround 300.00  Wait 0.00
  Job   1 -- Response: 300.00  Turnaround 600.00  Wait 300.00
  Job   2 -- Response: 600.00  Turnaround 900.00  Wait 600.00

  Average -- Response: 300.00  Turnaround 600.00  Wait 300.00
```

Response time grows linearly with job length. Each job's response time is the sum of the lengths of the jobs that run before it.

Q7:
Response time increases with the quantum, because each job waits longer for the jobs ahead of it to use their full slices. Once the quantum is at least as long as the jobs, RR behaves like FIFO.
The worst-case response time for N jobs with quantum q is: $(N - 1) \times q$